

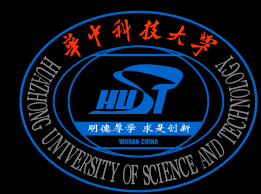
数字电路与逻辑设计

复习

张江山

zhangjs@hust.edu.cn

信息工程系



1 数字逻辑基础

- 1.1 数字电路与数字信号

- ◆ 占空比，周期，频率，脉宽

- 1.2 数制

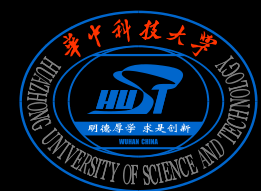
- ◆ 进位基数，数位权值，展开多项式
- ◆ 2、8、10、16 进制数表示方法
- ◆ 二 - 十进制转换（短除法 / 连乘法），二 - 十六 / 八进制转换

- 1.3 二进制数的算术运算

- ◆ 有符号数表示形式，求补操作符，数值加减运算，溢出判别

- 1.4 二进制代码

- ◆ 8421 码，2421 码，5421 码，余 3 码，余 3 循环码，格雷码
- ◆ ASCII 码



1 数字逻辑基础

- 1.5 二值逻辑变量与基本逻辑运算
 - ◆ 与，或，非，与非，或非，异或，同或
 - ◆ 二进制码 $\leftrightarrow$ 格雷码
- 1.6 逻辑函数及其表示方法
 - ◆ 真值表，逻辑表达式，逻辑图，波形图

- 2.1 逻辑代数

- ◆ 逻辑代数基本定律与恒等式
- ◆ 代入规则，反演规则，对偶规则
- ◆ 代数法化简、等式证明、表达式变换方法

- 2.2 逻辑函数的卡诺图化简法

- ◆ 最小项，最小项表达式
- ◆ 最大项，最大项表达式
- ◆ 卡诺图化简方法

● 3.1 MOS 逻辑门电路

- ◆ 输入和输出的高电平的下限和低电平上限值
- ◆ 噪声容限，传输延迟，功耗，延迟 - 功耗积，扇入、扇出数
- ◆ MOS 管静态特性（可变电阻区、截止区、饱和区）
- ◆ CMOS 管反相器，与非门，或非门（了解结构和特性）
- ◆ 输入、输出保护和缓冲电路的作用
- ◆ CMOS 漏极开路门 (OD)，三态门，传输门基本特性和原理

● 3.5 逻辑描述中的几个问题

- ◆ 正负逻辑变换，等效符号

● 3.6 逻辑门电路使用中的几个实际问题

- ◆ 门电路接口问题，多余输入端的处理

4 组合逻辑电路

● 4.1 组合逻辑电路的分析

- ◆ 输出逻辑表达式→化简→列真值表→画波形图→功能评述
- ◆ Verilog 对逻辑图的结构描述、逻辑表达式的数据流描述

● 4.2 组合逻辑电路的设计

- ◆ 确定 I/O 变量→列真值表→写逻辑表达式→化简→画电路
- ◆ 逻辑抽象
- ◆ Verilog 行为描述

● 4.3 组合逻辑电路中的竞争和冒险

- ◆ 竞争冒险概念，判断方法，消除方法

4 组合逻辑电路

● 4.4 常用组合逻辑集成电路

◆ 编码器： **CD4532** 优先编码器

◆ 译码器：

74HC139 二 - 四线译码器， **74HC138** 三 - 八线译码器，
74HC42 二 - 十进制译码器， **74HC4511** 七段显示译码器

◆ 数据选择器： **74HC151** 八选一选择器

◆ 数值比较器： **74HC85** 四位数值比较器

◆ 半加器、全加器、超前进位： **74LS283** 四位超前进位加法器

◆ 练习用 **Verilog** 描述上述集成电路功能

● 4.5 组合可编程电路

◆ 与、或阵列连线区的表示方法

5 锁存器与触发器

- 5.1 双稳态存储单元电路
- 5.2 锁存器
 - ◆ SR、D 锁存器原理，74HC373 八 D 锁存器
- 5.3 触发器的电路结构和工作原理
 - ◆ D 触发器原理，74HC74 双 D 触发器
- 5.4 触发器的逻辑功能
 - ◆ D、T、T'、SR、JK 功能表，特性方程
 - ◆ 功能转换

JK 触发器功能表

J K	Q^{n+1}	功能
0 0	Q	保持
0 1	0	置 0
1 0	1	置 1
1 1	$\bar{Q}$	翻转

$$Q^{n+1} = J \bar{Q} + \bar{K} Q$$

- 6.1 时序逻辑电路的基本概念
- 6.2 同步时序逻辑电路的分析
 - ◆ 代数法：写输出、激励、状态方程组→列状态表→画状态图或时序图→描述功能
 - ◆ 表格法：写输出、激励→列次态真值表→列状态表→画状态图或时序图→描述功能
 - ◆ 米利型，穆尔型区别
- 6.3 同步时序逻辑电路的设计
 - ◆ 确定 I/O 变量→画原始状态图→状态化简→状态编码→确定触发器→写输出、激励方程→画逻辑图→检查自启动
- 6.4 异步时序逻辑电路的分析
 - ◆ 写时钟，输出和激励方程→列次态方程 / 次态真值表→列状态表→画状态图 / 时序图→描述功能→检查自启动

● 6.5 若干典型的时序逻辑集成电路

◆ 寄存器

- 74HC374 八位寄存器
- 74HC164 八位移位寄存器
- 74HC194 四位双向移位寄存器

◆ 计数器

- 74HC393 异步四位二进制计数器
- 74LVC161 四位二进制计数器
- 74HC390 四位二 - 十进制计数器
- 任意进制计数器的实现方法
- 74HC4017 十进制扭环形计数器

◆ 练习用 Verilog 描述上述集成电路的功能

● 6.7 时序逻辑可编程逻辑器件（GAL）

● 7.1 只读存储器

- ◆ 了解 ROM 基本结构， AT27C010 ROM 芯片的典型时序
- ◆ 掌握 ROM 实现组合逻辑电路功能的方法

● 7.2 随机存取存储器

- ◆ 了解 SRAM， SSRAM， DRAM 组成原理
- ◆ 掌握容量扩展方法

● 8.1 CPLD

- ◆ 与、或阵列实现组合逻辑
- ◆ 触发器可实现时序逻辑

● 8.2 FPGA

- ◆ LUT 实现组合逻辑， LUT 的组成
- ◆ 触发器可实现时序逻辑

9 脉冲波形的变换与产生

● 9.1 单稳态触发器

- ◆ 了解原理
- ◆ 了解影响脉宽和最大频率的因素
- ◆ 用途：定时、延迟、频率计等

● 9.2 施密特触发器

- ◆ 了解原理，电平敏感
- ◆ 了解影响正、负阈值的因素
- ◆ 用途：波形整形、抗干扰等

● 9.3 多谐振荡器

- ◆ 了解原理
- ◆ 了解影响振荡周期的因素

● 10.1 DAC

- ◆ 倒 T 形电阻网络 D/A 转换器原理
- ◆ 参考电压与量程范围的关系
- ◆ AD7533 集成 D/A 芯片的应用
- ◆ 单极性和双极性
- ◆ 影响指标的因素

● 10.2 ADC

- ◆ 并行比较型，逐次比较型，双积分型原理
- ◆ 参考电压和量程范围的关系
- ◆ ADC0809 集成 A/D 芯片的应用
- ◆ 影响指标的因素

● Verilog HDL 基础语法

- ◆ 定义模块：`module`, `endmodule`
- ◆ 输入 / 输出端口类型：`input`, `output`, `inout`
- ◆ 参数定义：`parameter`
- ◆ 变量类型：`wire`, `reg`, `integer`, `real`, `time`
- ◆ 运算符及优先级：优先级按由高至低顺序，行内分号左高右低
 - ◆ 单目运算：`+`, `-`, `!`, `~`, `&`, `|`, `^`, `~&`, `~|`, `~^`
 - ◆ 算术运算：`**`; `*`, `/`, `%`; `+`, `-`
 - ◆ 移位运算：`<<`, `>>`, `<<<`, `>>>`
 - ◆ 关系运算：`>`, `>=`, `<`, `<=`; `==`, `!=`, `===`, `!==`
 - ◆ 位逻辑运算：`&`; `^`, `~^`; `|`
 - ◆ 逻辑运算：`&&`; `||`
 - ◆ 条件运算：`?:`
 - ◆ 拼接与复制运算：`{}`, `{{}}`

- 门级建模：逻辑图描述
 - ◆ 多输入门：`and`, `or`, `nand`, `nor`, `xor`, `xnor`
 - ◆ 多输出门：`not`, `buf`
 - ◆ 三态门：`notif1`, `notif0`, `bufif1`, `bufif0`
- 数据流建模：逻辑 / 算术 / 条件表达式描述
 - ◆ 连续赋值语句：`assign`
- 行为建模：信号变化行为描述
 - ◆ 行为描述语句：`always`, `initial`
 - 过程语句块：`begin`, `end`
 - 条件语句：`if`, `else if`, `else`
 - 分支语句：`case`, `casex`, `casez`, `endcase`
 - 循环语句：`for`, `while`
 - 过程赋值：`=`, `<=`
 - 敏感事件：`posedge`, `negedge`

- 数据流建模与仿真
 - ◆ 编码器：类似 CD4532 编码器模块的数据流建模
 - ◆ 译码器：类似 139 / 138 译码器模块的数据流建模
 - ◆ 选择器：类似 151 选择器模块的数据流建模
- 分层结构建模与仿真
 - ◆ 比较器：类似 85 比较器模块的分层结构建模
 - ◆ 加法器：类似 283 加法器模块的分层结构建模
- 行为建模与仿真
 - ◆ 锁存器：各种锁存器模块的行为建模
 - ◆ 触发器：各种触发器模块的行为建模
 - ◆ 寄存器：类似 194 多功能双向移位寄存器模块的行为建模
 - ◆ 计数器：类似 161 计数器模块的行为建模
 - ◆ 状态图：米利型 / 摩尔型状态图模块的行为建模

● 例：分析 Verilog 描述的 test 模块功能

```

module test(output[3:0] Q, input CP, _RST);
    shiftReg U1(Q, {~(_RST&(~(&Q))), 1'b1}, 1'b0, Q[3], CP, 1'b1, 4'hE);
endmodule

module shiftReg(output reg[3:0] Q,
                input [1:0] S, input Dsd, Dsu,
                input CP, _CR, input [3:0] D);
    always @(posedge CP, negedge _CR)
        if (~_CR) Q <= 4'b0000;
        else
            case (S)
                2'b00: Q <= Q;
                2'b01: Q <= { Q[2:0], Dsu };
                2'b10: Q <= { Dsd, Q[3:1] };
                2'b11: Q <= D;
            endcase
        endmodule
    
```

